In MongoDB, **failover and consistency are primarily implemented through replica sets, elections, the oplog, and configurable read/write concerns**.

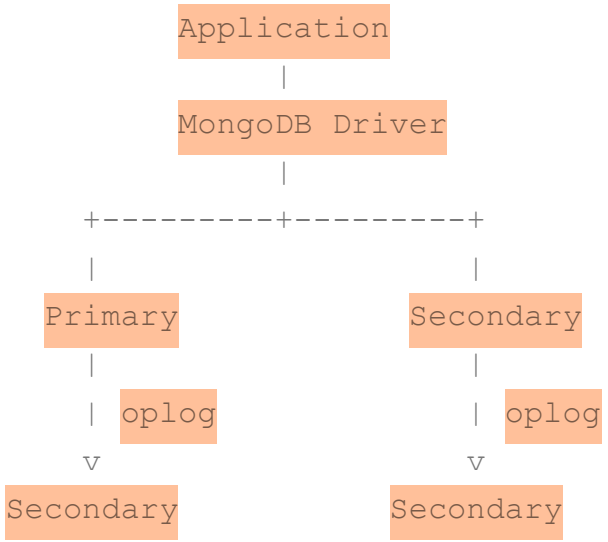
The most important idea is that MongoDB separates:

- **Replication** — keeping multiple copies of data.
- **Election/failover** — choosing a new primary when the current primary fails.
- **Consistency** — controlling when writes are acknowledged and which copies reads can come from.

---

# 1. Basic architecture

A MongoDB replica set typically looks like:




There is normally:

- **one primary**
- **one or more secondaries**
- optionally **arbiter** nodes in certain topologies

The primary accepts writes.

Secondaries replicate the primary's operations.

---

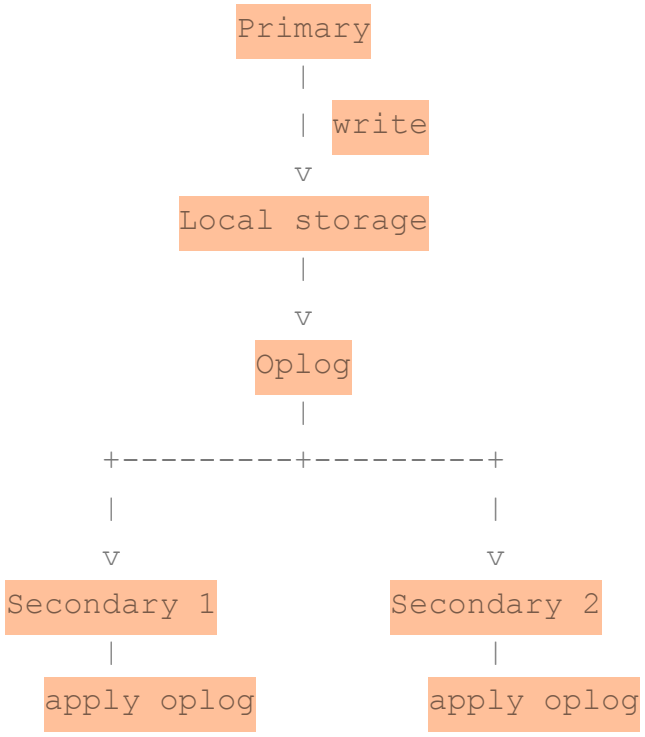
# 2. How replication works

When the application writes:

```
db.orders.insertOne({
    orderId: 1001,
    customerId: 42
})
```

the primary performs the write and records the operation in its **oplog**.

Conceptually:




The secondaries continuously replicate operations from the primary's oplog and apply them locally.

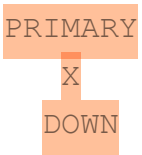
The oplog is therefore fundamental to both:

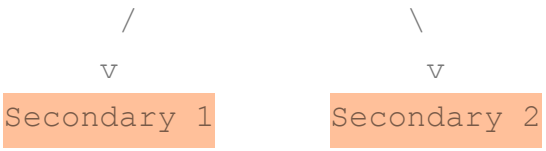
- replication
- recovery/failover

---

# 3. What happens when the primary fails?

Suppose:

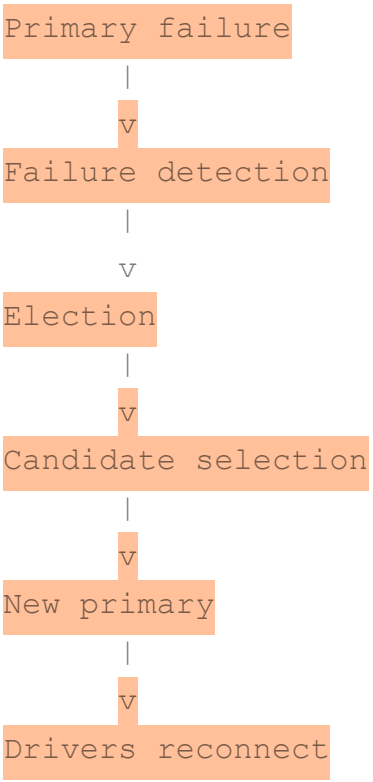




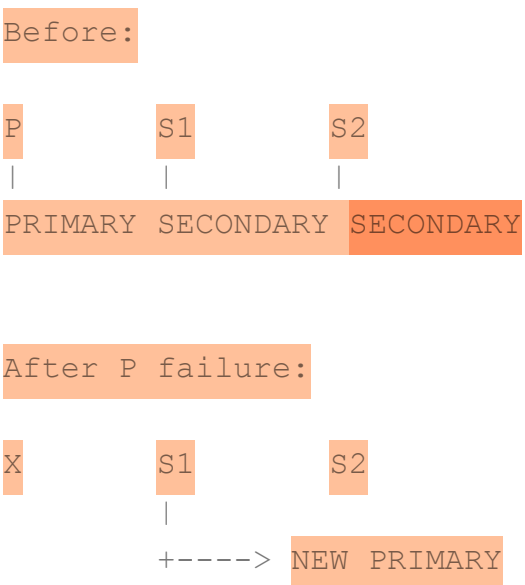

The remaining eligible members detect that the primary is unavailable.

They initiate an **election**.

Conceptually:




For example:



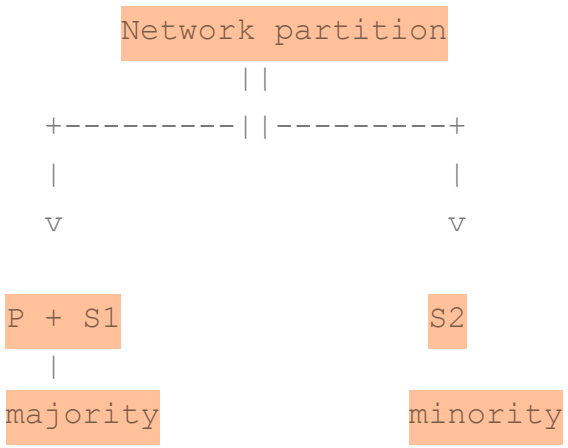
MongoDB's replica-set election protocol ensures that normally only one member becomes primary for a given replica set.

---

# 4. Why don't two nodes become primary?

This is the **split-brain** problem.

Imagine a network partition:




The majority side can continue operating.

The minority side cannot simply declare itself primary.

This is based on **majority voting**.

For a three-member replica set:

```
Members = 3
Majority = 2
```

Therefore:

```
P + S1 -> majority
S2     -> minority
```

Only the side with a voting majority can elect a primary.

This is why an odd number of voting members is often useful.

---

# 5. Election isn't just "pick any secondary"

The election process considers member state and eligibility, including factors such as:

- voting configuration
- priority
- replication state
- configuration
- topology
- whether a member is sufficiently caught up

You can influence preferred primary placement with member priorities.

For example:

```
Region A
  Primary candidate priority = high

Region B
  Secondary priority = lower

Region C
  Secondary priority = lower
```

This lets you express preferences such as:

> Prefer the primary in Region A, but fail over to another region if necessary.

---

# 6. What does the application experience?

Suppose:

```
Client
  |
  v
Primary
```

and the primary dies.

The MongoDB driver detects the failure and discovers the new topology:

```
Old:

P -> S1 -> S2

New:

X -> S1 -> S2
     ^
     |
   PRIMARY
```

The driver reconnects to the new primary.

Modern MongoDB drivers support automatic server discovery and monitoring, so applications generally don't need to manually determine which node is primary.

There is nevertheless a **failover window** during which requests can fail or need retrying.

Therefore application-level retry handling is important.

---

# 7. Failover does NOT mean zero downtime

This is an important interview distinction.

Suppose:

```
t0        Primary healthy
t1        Primary crashes
t2        Failure detected
t3        Election
t4        New primary
t5        Application reconnects
```

During some portion of:

```
t1 -> t5
```

writes may fail.

So the correct statement is:

> MongoDB provides automatic failover, but failover has a finite interruption window.

Your system's SLA must therefore account for election/recovery time.

---

# 8. Now the important part: consistency

MongoDB provides configurable consistency rather than one single consistency model.

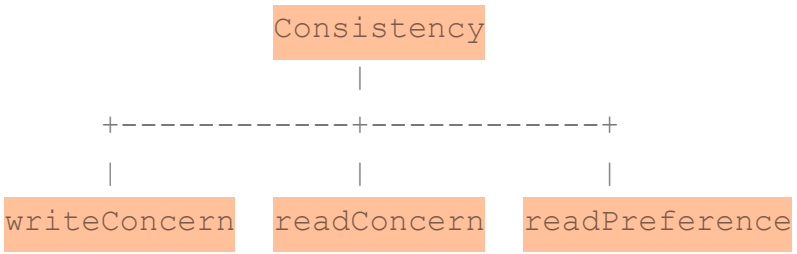
Two major controls are:

```
writeConcern
readConcern
```

And another important control is:

```
readPreference
```

Think of them as:



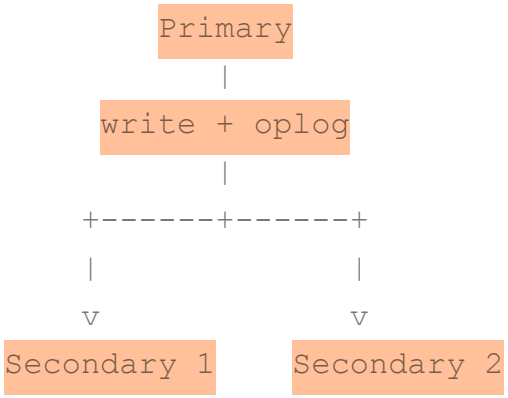

---

# 9. Write concern

Consider:

```
db.orders.insertOne(
    { orderId: 1 },
    { writeConcern: { w: "majority" } }
)
```

Conceptually:




The write isn't considered successfully acknowledged merely because the primary accepted it locally.

With:

```
w: "majority"
```

MongoDB waits for acknowledgement from a **majority of voting members** according to the configured write-concern semantics.

For a three-member replica set:

```
P + S1 = majority
```

Once the required majority has acknowledged the operation, the client receives success.

---

# 10. Why majority writes matter for failover

This is the key relationship.

Suppose:

```
3 members

P
S1
S2
```

Client writes:

```
X = order 1001
```

and waits for:

```
w: majority
```

Suppose:

```
P -> X
S1 -> X
S2 -> not yet X
```

Now P dies:

```
P = DEAD
```

S1 still contains X.

Therefore S1 can become the new primary without losing the acknowledged write.

This gives us the fundamental property:

> An acknowledged majority write is replicated to enough voting members that an election can preserve it across an ordinary primary failover.

---

# 11. What if we use `w: 1`?

Consider:

```
writeConcern = { w: 1 }
```

The primary can acknowledge the write after its own successful write.

Suppose:

```
P -> X
S1 -> old state
```

```
S2 -> old state
```

Client receives:

```
SUCCESS
```

Then P crashes before secondaries replicate X.

An election could select a secondary that doesn't have X.

Therefore:

```
Client acknowledged X
        |
        v
Primary crashes
        |
        v
New primary
        |
        v
X may not be present
```

This is one reason `w: 1` provides weaker durability than majority acknowledgement.

---

# 12. Write concern ≠ read concern

This distinction is extremely important.

`writeConcern` answers:

> **When can I tell the client that the write has been sufficiently persisted/replicated?**

`readConcern` answers:

> **What guarantees should a read have about the data it observes?**

They're separate mechanisms.

---

# 13. Read preference

Now consider where reads go.

MongoDB supports policies such as:

```
primary
primaryPreferred
secondary
secondaryPreferred
nearest
```

For example:

```
readPreference = primary
```

means:

```
Application
     |
     v
  Primary
```

while:

```
readPreference = secondaryPreferred
```

can allow reads to go to secondaries.

This creates a trade-off:

```
Secondary reads
        |
        +----> more read capacity
        |
        +----> potentially stale data
```

---

# 14. The stale-read problem

Suppose:

```
t0:
Primary:   balance = 100
Secondary: balance = 100
```

Client executes:

```
balance = 50
```

Primary updates immediately:

```
Primary:   50
Secondary: 100
```

Before replication catches up, a read routed to the secondary returns:

```
100
```

So:

```
write -> Primary
read  -> Secondary
```

can produce:

```
read-your-write violation
```

unless the appropriate consistency controls are used.

---

# 15. Read concern

MongoDB provides read concerns that let applications express stronger requirements.

One particularly important level is:

```
majority
```

Conceptually, a majority read is concerned with data that has been acknowledged by a majority of voting members.

Another important concept is:

```
"snapshot"
```

which provides transactional snapshot semantics in supported contexts.

And:

```
"linearizable"
```

can provide a very strong guarantee for eligible reads from the primary, at additional latency/availability cost.

You should not casually configure the strongest level everywhere.

---

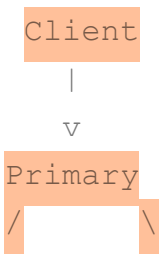
# 16. `majority` read + `majority` write

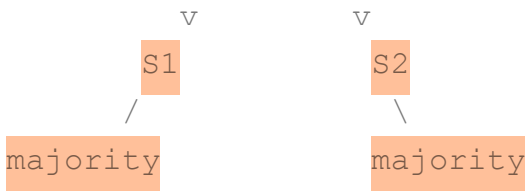
A common durability-oriented configuration is conceptually:

```
Writes:
    w: majority

Reads:
    readConcern: majority
```

The resulting architecture looks like:






The important property is that acknowledged data has majority durability and reads can avoid observing data that isn't sufficiently committed according to the read concern.

This is much stronger than:

```
w: 1
+
secondaryPreferred
```

---

# 17. What is the "majority commit point"?

This is a very useful concept for understanding MongoDB consistency.

Suppose:

```
Primary oplog:

A B C D E F G
```

and secondaries have:

```
S1: A B C D E
S2: A B C D
```

The majority has reached:

```
D
```

Conceptually:

```
A B C D
^     ^
|     |
```

```
old      majority
         commit point
```

Operations up through the majority commit point have sufficient replication acknowledgement to satisfy majority-based durability semantics.

As secondaries progress:

```
S1 -> F
S2 -> E
```

the majority commit point advances.

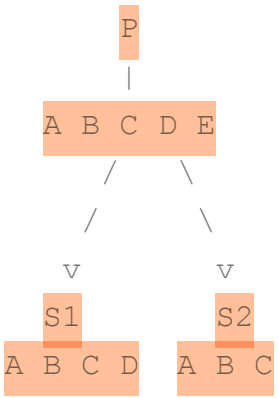
This is a core mechanism behind majority read/write semantics.

---

# 18. Failover + majority commit

Now combine the two concepts.

Before failure:




Suppose:

```
D = majority committed
E = not majority committed
```

Primary fails:

```
P -> DEAD
```

Election chooses S1:

```
S1 = NEW PRIMARY
```

S1 contains:

```
A B C D
```

Therefore the majority-committed state survives.

This is why majority acknowledgement and elections are closely connected.

---

# 19. What happens to writes that weren't majority committed?

This is another subtle interview point.

Suppose:

```
P:
A B C D E F

S1:
A B C D

S2:
A B C D
```

`E` and `F` were not replicated to a majority.

P dies.

S1 becomes primary.

The cluster's durable state may therefore effectively roll back those uncommitted operations from the perspective of the new primary.

This is called a **rollback**.

So:

```
majority acknowledged
        |
        v
survives normal failover

not majority acknowledged
        |
        v
may be rolled back
```

This is one of the most important reasons applications should understand their write concern.

---

# 20. Rollback

Conceptually:

```
Old Primary:

A B C D E F
        ^^^
      divergent

New Primary:

A B C D
```

The old primary, if it rejoins, reconciles its divergent history with the new primary.

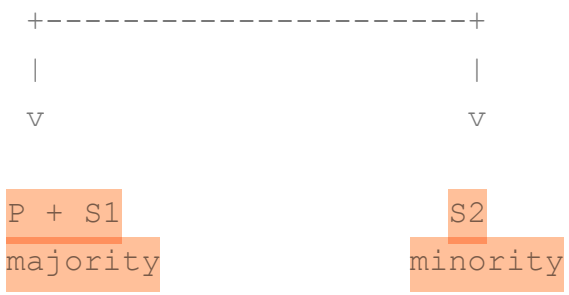
The application should not assume that an operation acknowledged only locally has the same durability guarantee as a majority-acknowledged operation.

---

# 21. What about network partitions?

Consider:

```
            Network partition
```



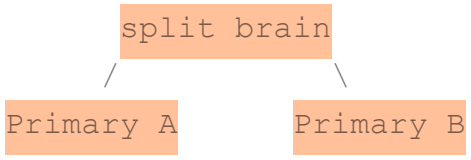

P + S1 have two votes out of three.

They can maintain/elect a primary.

S2 has only one vote.

It cannot independently establish itself as primary.

This prevents the classic:




scenario.

The majority mechanism is therefore both:

- a **consistency mechanism**
- a **split-brain prevention mechanism**

---

# 22. Why three nodes are common

For:

```
N = 3
```

you need:

```
majority = 2
```

You can tolerate:

```
1 failure
```

while retaining a majority.

For:

```
N = 5
```

you need:

```
majority = 3
```

and can tolerate:

```
2 failures
```

The general rule is:

```
majority = floor(N / 2) + 1
```

But don't simply keep adding voting members. Larger voting configurations increase election/replication complexity and operational cost.

---

# 23. Availability vs consistency trade-off

Here's where system-design reasoning becomes important.

Suppose a three-node replica set has:

```
P
S1
S2
```

and S1/S2 become unavailable.

You now have:

```
P only
```

No majority exists.

With majority-oriented write concern, writes may no longer be acknowledged.

That's deliberate.

You are effectively choosing:

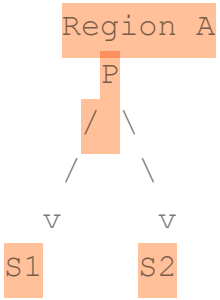
```
Consistency / durability
          >
Availability
```

rather than accepting writes that cannot achieve the desired durability guarantee.

---

# 24. Geographic failure

Suppose you deploy:




If Region A fails completely:

```
Region A
   X
```

you lose the entire replica set.

A better topology is:

```
Region A              Region B              Region C

  Primary               Secondary             Secondary
```



Now a regional failure can still leave enough voting members for an election, depending on the exact topology and failure.

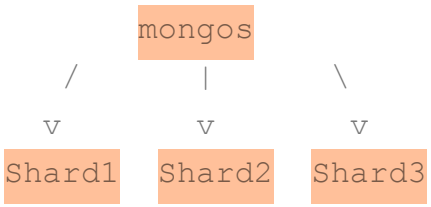
This is why **failure-domain-aware placement** is essential.

---

# 25. Sharded MongoDB: failover happens per shard

This becomes particularly important when MongoDB is sharded.

Recall:




Each shard is normally a replica set:

```
Shard 1:

P1
S1
S2


Shard 2:

P2
S3
S4


Shard 3:

P3
```

```
S5
S6
```

If:

```
P2
```

fails:

```
Shard 2:

X
S3
S4
```

S3/S4 conduct an election.

The other shards don't need to elect new primaries.

So failover is **localized to the affected replica set**.

---

# 26. `mongos` and failover

Applications generally connect through MongoDB drivers that understand the cluster topology.

Conceptually:

```
Application
     |
     v
MongoDB Driver
     |
     v
  mongos
  /  |  \
 v   v   v
S1   S2   S3
```

If the primary of S2 changes:

```
S2:
old P -> new P
```

the driver/router refreshes topology information and routes subsequent writes appropriately.

The application therefore shouldn't hard-code:

```
shard2-primary.example.com
```

as its permanent write endpoint.

---

# 27. Consistency across shards is harder

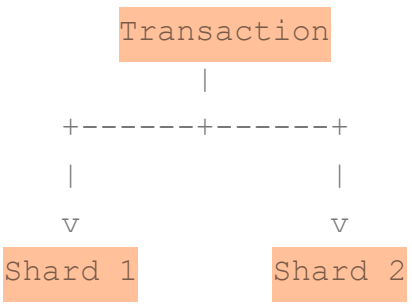
Here's an important distinction:

```
Replica-set consistency
        ≠
cross-shard transaction consistency
```

Suppose an operation touches:

```
Shard 1
Shard 2
```

Now we have distributed coordination.




MongoDB supports distributed transactions, but they introduce additional coordination and latency.

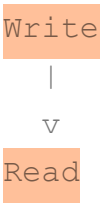
Therefore a good schema/shard-key design tries to keep highly coupled operations together when possible.

---

# 28. Sessions and causal consistency

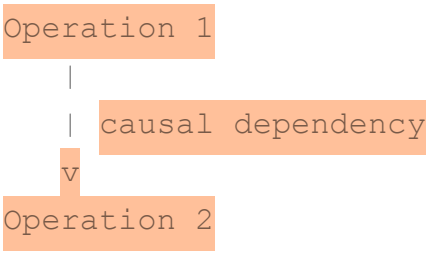
MongoDB also supports **logical sessions** and causal consistency mechanisms.

This is useful when an application performs:




and needs the read to observe causally related operations.

Conceptually:



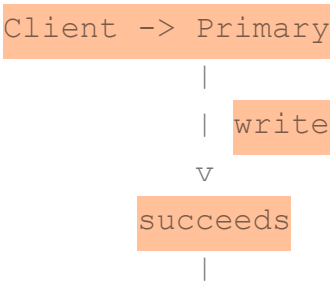

The client/session carries causal ordering information so that the system can maintain the required ordering across operations.

This is especially useful in distributed deployments where reads and writes may involve different members.

---

# 29. Retryable writes and failover

There's another important failure mode:



```
        X
network failure
```

The client doesn't know whether the operation succeeded.

It sees:

```
NETWORK ERROR
```

Did MongoDB execute the write?

Possibly.

If the client blindly retries:

```
retry(write)
```

you could potentially execute an operation twice.

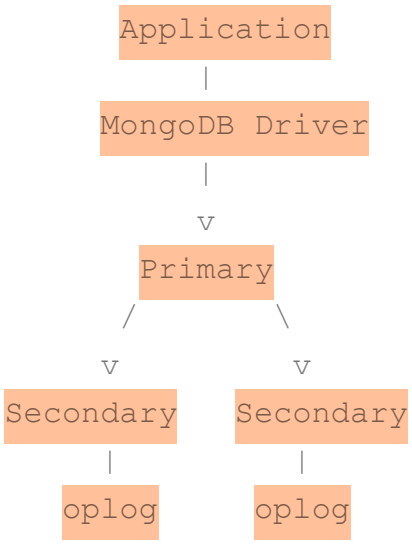
MongoDB supports **retryable writes** for supported operations, allowing the driver/server to identify retries and avoid unintended duplicate execution under the supported semantics.

This is extremely important for robust application design.

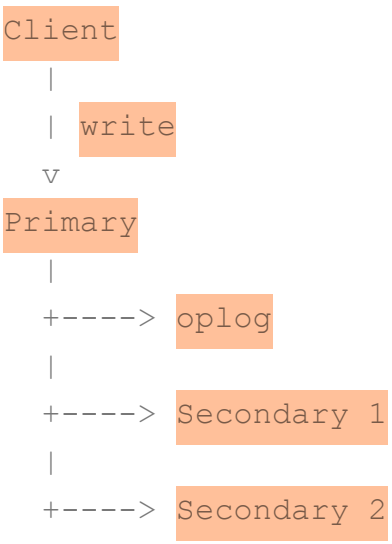
---

# 30. The complete failure path

Put everything together:



## Normal write



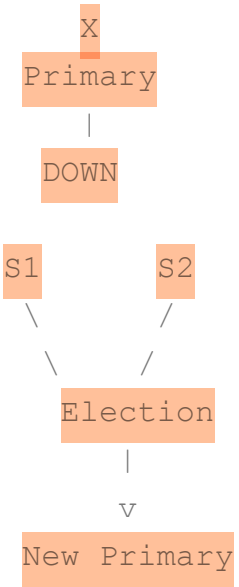

With:

```
w: majority
```

the client waits for sufficient replication acknowledgement.

---

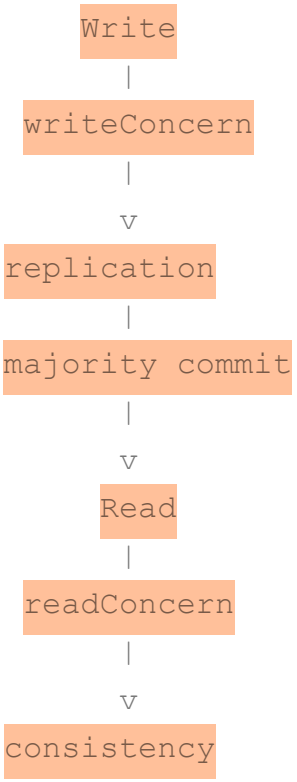
## Primary failure




Drivers discover the new primary.

---

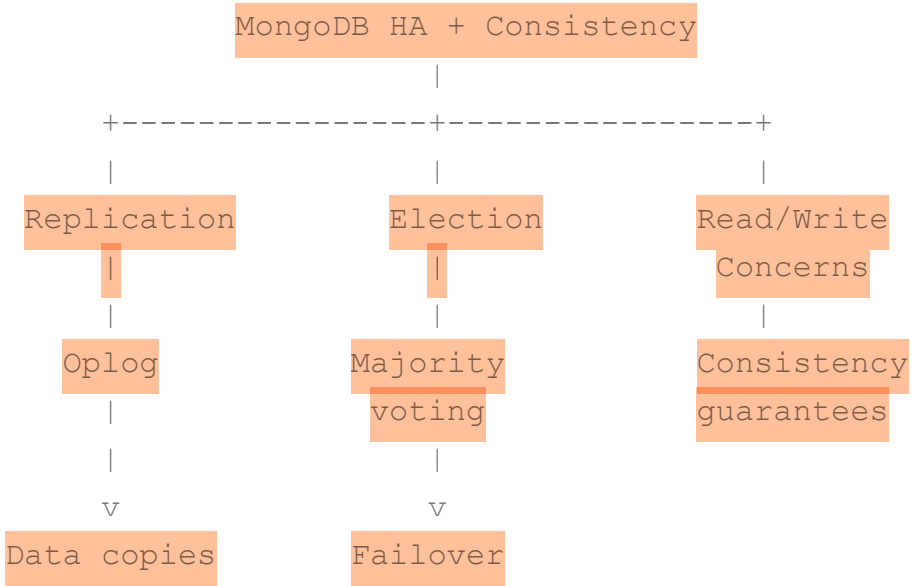
## Consistency



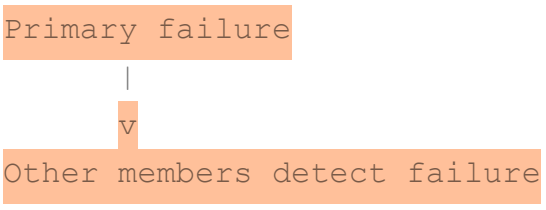

---

# 31. The interview-ready model

I recommend remembering this four-layer model:




And the failure sequence:



```
      |
      v
Election
      |
      v
Candidate with majority
      |
      v
New primary
      |
      v
Drivers rediscover topology
      |
      v
Writes resume
```

While the consistency sequence is:

```
Client write
     |
     v
writeConcern
     |
     v
Replication
     |
     v
Majority commit point
     |
     v
Acknowledgement
```

```
Client read
     |
     v
readPreference
     |
     v
Selected node
     |
     v
readConcern
     |
```

```
v
Consistency guarantee
```

## The five things I'd emphasize in an interview

1. **Replica sets provide HA through redundant copies and automatic elections.**
2. **Elections use majority voting to prevent split brain.**
3. **The oplog provides the replication mechanism from primary to secondaries.**
4. `writeConcern` **, especially** `w: "majority"` **, determines how much replication is required before acknowledging a write.**
5. `readConcern` **+** `readPreference` **determine what data a read can observe and which replica it reads from.**

The deeper insight is:

> **MongoDB does not get consistency merely because it has replicas. Replication provides copies; the election protocol provides failover; majority commit semantics provide durable ordering; and read/write concerns let the application choose its consistency–latency–availability trade-off.**

For a MAANG+ system-design interview, the next natural layer is **exactly what happens during a primary failure at the oplog/term/election level, including election terms, rollback, majority commit point,** `w: "majority"` **, and how a client behaves when the network fails after the server has accepted a write.**